Vue router

Tema 4. Unidad 4

Ejercicios

Universitat Oberta de Catalunya

Index

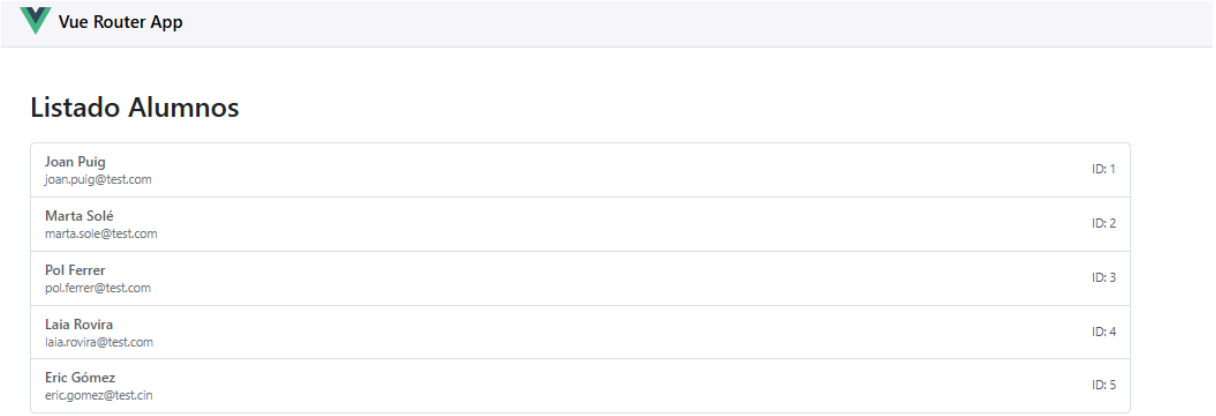
Ejercicio 1: Listado - detalle 2
Ejercicio 2: Gestión de filtros 4
Ejercicio 3: Control de acceso por roles 5

Ejercicio 1: Listado - detalle

Vamos a practicar el enrutamiento con una pequeña aplicación simulando un listado de alumnos que nos permita ver su detalle. Necesitamos definir la estructura mediante una interfaz para nuestros objetos Student y un array de datos estáticos.
Para definir el array de datos la estructura tiene de contener:

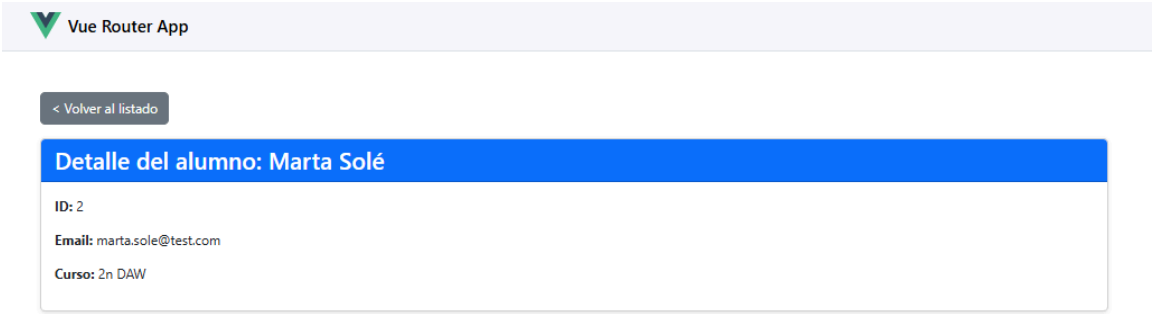
```
export interface Student {
    id: number;
    name: string;
    email: string;
    course: string;
}
```

Con esta definición ya podemos crear datos ficticios en el array que nos permite mostrarlos por pantalla como podemos ver en la imagen T4U4EJ1_IMG1.



T4U4EJ1_IMG1

Al clicar en un alumno nos tiene que mostrar otra página distinta con el detalle del alumno como vemos en la imagen T4U4EJ1_IMG2, donde podemos ver todos los datos del alumno. Además tenemos que añadir un botón para poder volver al listado.

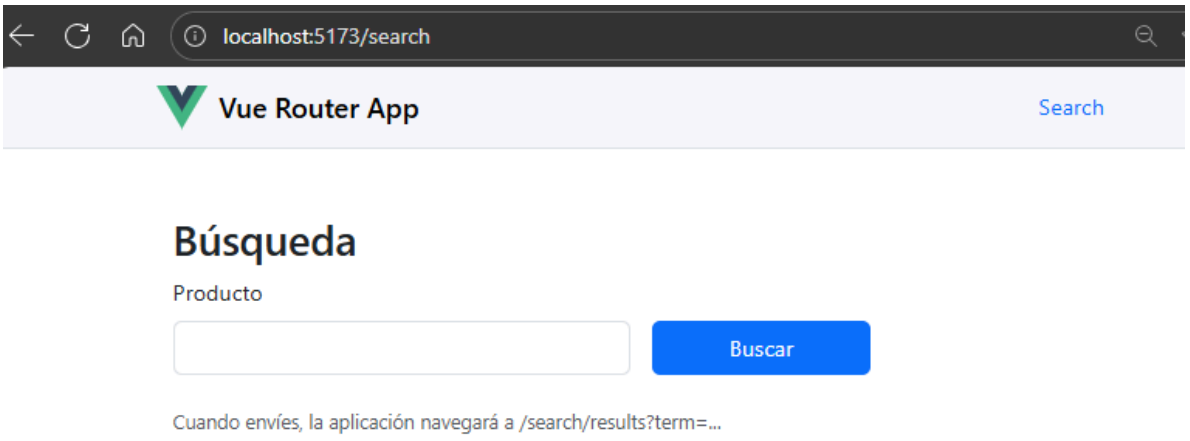


T4U4EJ1_IMG2

Recordad configurar el router para poder recibir el id del alumno seleccionado por la ruta. También podéis ayudaros del framework Bootstrap para los estilos.

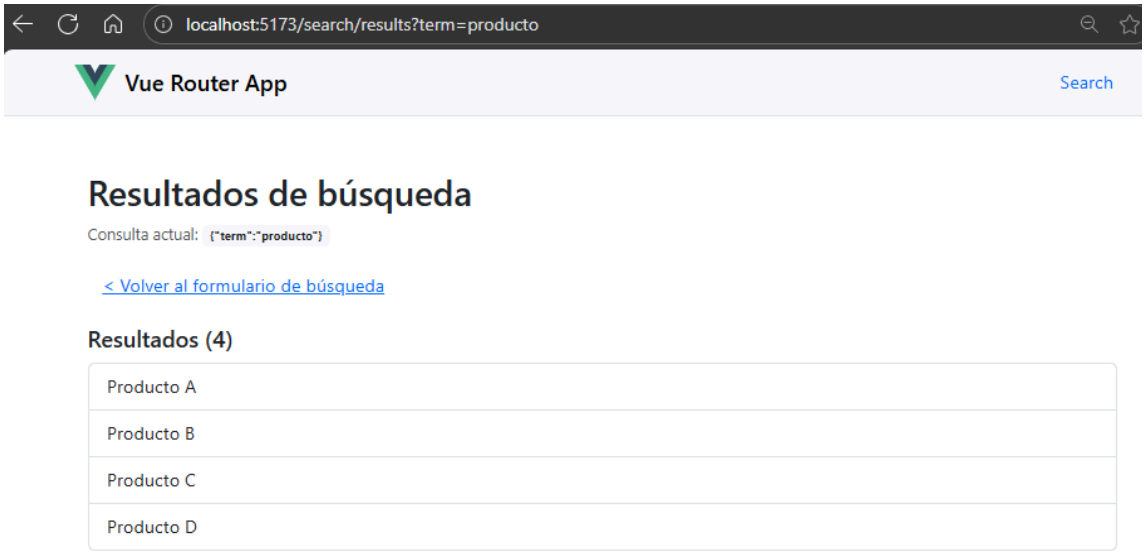
Ejercicio 2: Gestión de filtros

Como hemos visto podemos utilizar los parámetros de la ruta de forma dinámica. En este ejercicio vamos a practicar con los query parameters de la url para filtrar un listado de productos. Esto no permitirá que el usuario pueda compartir la url de búsqueda con los filtros aplicados.
Para realizar el ejercicio hemos de crear dos páginas, una primera con un pequeño formulario que nos permita introducir un texto. Este texto será el que enviaremos por la url (no necesitaremos props true en el router). En la página de destino recogeremos este parámetro y lo usaremos para filtrar el listado de productos y mostrar los correspondientes.
Para practicar la navegación programática (router.push) en la página de resultado pondremos un botón para volver al formulario de búsqueda. Y para mejorar la experiencia del usuario y seguir practicando con las rutas al volver tenemos que recuperar de la url el valor introducido previamente y volver a mostrarlo en el formulario de búsqueda.
Otra validación que podemos realizar es recargar la página de resultados con los parámetros en la url para verificar el funcionamiento.
En la imagen T4U4EJ2_IMG1 vemos el formulario en el estado inicial.

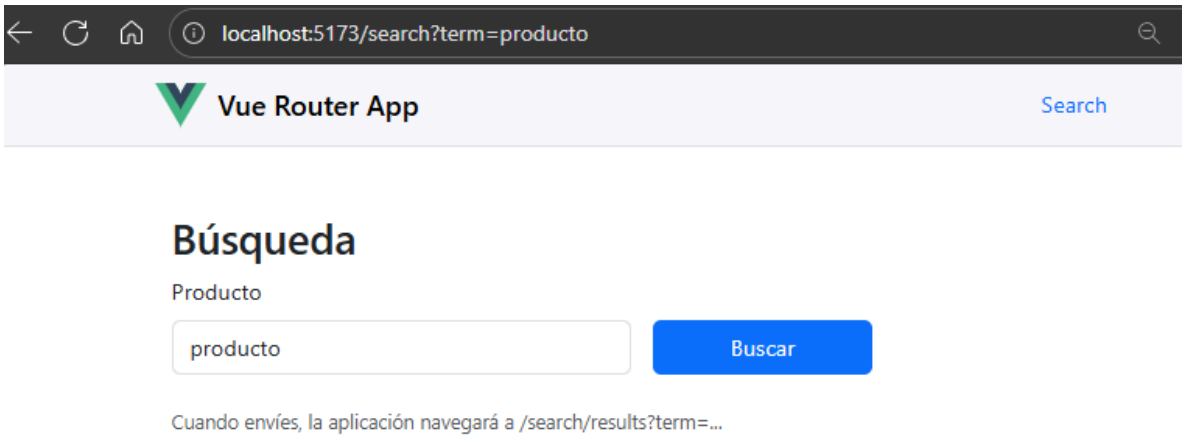


T4U4EJ2_IMG1

Supongamos que realizamos una búsqueda con el nombre “producto”, en la imagen T4U4EJ2_IMG2 vemos el resultado.



T4U4EJ2_IMG2



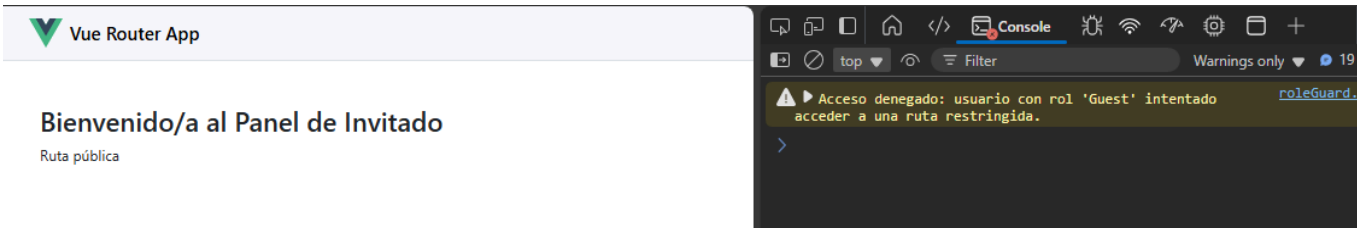
T4U4EJ2_IMG3

Ejercicio 3: Control de acceso por roles

El objetivo de este ejercicio es utilizar los guards para acceder a distintas páginas simulando diferentes roles para el usuario.
Para la simulación del usuario crearemos un servicio o un modelo con la siguiente estructura:

```
export interface UserState {
  isAuthenticated: boolean;
  role: 'Guest' | 'Editor' | 'Admin';
}
```

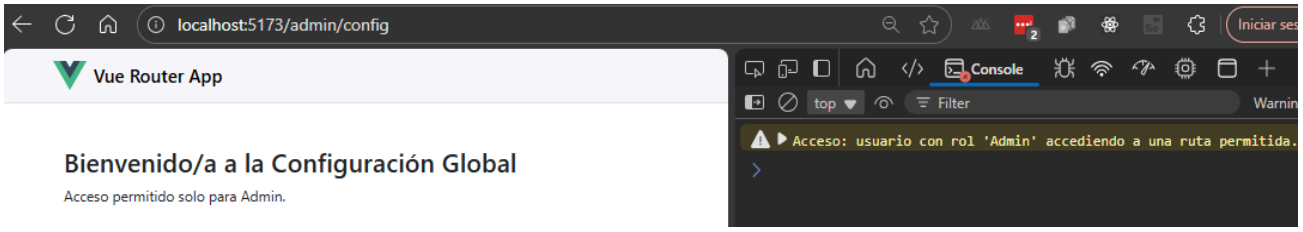
en el mismo servicio crearemos un función para recuperar el usuario que nos permita modificar el rol y si está autenticado o no. Podemos llamar a la función *getCurrentUser*. Definiremos la función *roleGuard*, que acepte un array de los roles permitidos y verifique si el usuario simulado pertenece a uno de los roles. Si no cumple con el rol y no está autenticado se redirige a la página principal. Mostraremos un mensaje por la consola mostrando el motivo y el rol del usuario (ver imagen T4U4EJ3_IMG1).



T4U4EJ3_IMG1

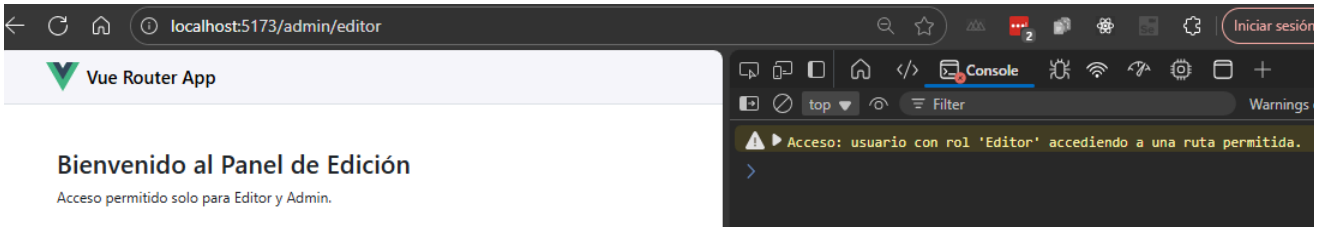
Para ver el comportamiento con los distintos roles crearemos tres vistas en tres componentes distintos para asignarlos en rutas diferentes según los roles.

- Componente AdminConfig.vue: se accede con el rol ‘Admin’, muestra por pantalla mensaje de bienvenida para el administrador. Ruta /admin/config.
-



T4U4EJ1_IMG2

- Componente EditorPanel.vue: se accede con el rol ‘Editor’ y ‘Admin’. Ruta /admin/editor.



T4U4EJ1_IMG2

- Componente Guest.vue: todos los usuarios que no están autorizados. Ruta /.